

Guía de Diseño v2.0

Valdris: El Núcleo Profundo

Documento operativo de implementación · Complemento a la Guía de Proyecto v3

Proyecto	Valdris: El Núcleo Profundo — Juego por Turnos JavaFX
Grupo	H12GEXTRA — Marcos Castro · Ventura Pacheco · Marino Rodríguez
Versión	v2.0 — Fórmula de combate actualizada · Diagrama de flujo estático añadido
Compañero	Guía de Proyecto v3.0 (narrativa, arquitectura, advertencias técnicas)

Cambios respecto a v1.0
- Fórmula de combate actualizada: $\text{daño} = \max(0, \text{ataque} \cdot (\text{rand}[0.5-1.5]) - \text{defensa})$. Rango restringido para evitar fallos totales injustos. - División de trabajo actualizada con nombres reales: Marino, Pacheco, Castro. - Diagrama de flujo de turno añadido como figura estática en Sección 1.0 (también disponible interactivo en la conversación).

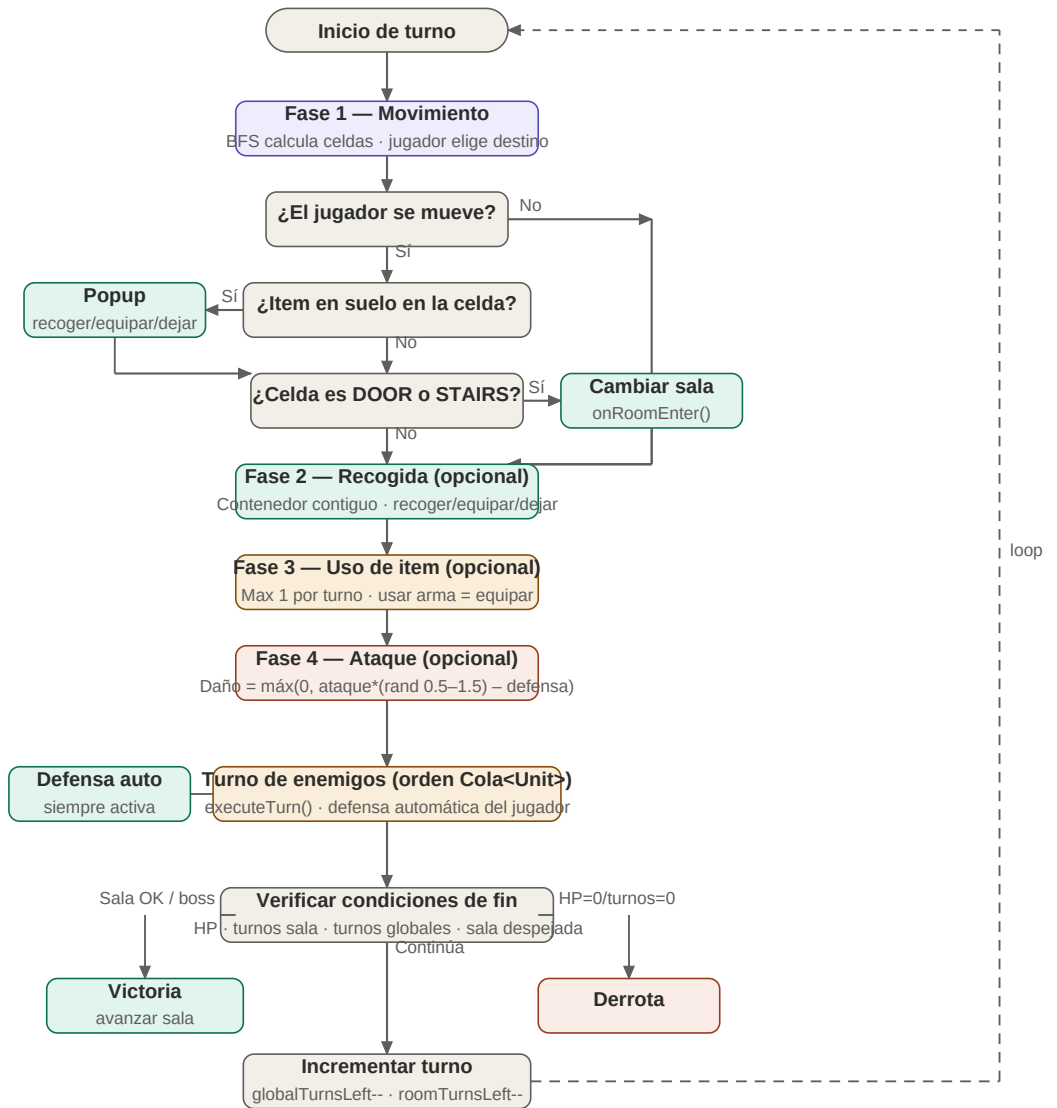
Punto 1 — Sistema de Turnos Completo

Esta sección define con precisión todo lo relativo al sistema de turnos del juego: qué puede hacer el jugador en su turno, en qué orden, qué restricciones aplican, cómo actúan los enemigos, y qué condiciones terminan la partida. Es la referencia definitiva para implementar TurnManager.

1.0 Diagrama de flujo completo de un turno

El siguiente diagrama muestra el flujo completo de un turno: las cuatro fases del jugador en orden fijo, el turno de los enemigos, la verificación de condiciones y el loop de vuelta al inicio. Las flechas punteadas representan el loop al inicio del siguiente turno.

■ FASE JUGADOR



■ FASE ENEMIGOS

■ FIN DE TURNO

■ Fase jugador

■ Evento / acción especial

■ Turno enemigos

■ Combate / derrota

■ Control de flujo

1.1 Acciones disponibles del jugador

En un turno el jugador puede realizar como máximo una acción de cada tipo, en el orden fijo definido a continuación. Ninguna acción es obligatoria — el jugador puede ceder el turno sin hacer nada.

Acción	Máximo por turno	Restricción	Notas
Movimiento	1	Siempre va primero si se realiza	BFS calcula celdas alcanzables. Consumir movePoints.
Recogida	1	Fase 1 (item suelo) o Fase 2 (contenedor contiguo)	Item en suelo: popup automático al pisar. Contenedor: acción manual en Fase 2.
Uso de item	1	Solo items del inventario. Usar arma = equipar (no atacar)	Pociones, items especiales. No se puede usar poción Y atacar con arma en el mismo turno.
Ataque	1	Solo si hay enemigo en rango del arma equipada	Sin arma: rango 1, daño base. Con arma: rango y daño del arma.
Ceder turno	—	En cualquier momento	El jugador no hace nada. Pasa al turno enemigo.

Caso especial — usar arma vs atacar con arma

- Usar una espada del inventario = equiparla. Equipar NO es atacar.
- Atacar con la espada YA equipada SÍ es la acción de ataque (Fase 4).
- Flujo correcto: recoger espada (Fase 2) → equipada automáticamente → atacar con ella (Fase 4).
- El jugador PUEDE recoger una espada y atacar con ella en el mismo turno.
- El jugador NO PUEDE usar una poción Y atacar con arma en el mismo turno.

1.2 Orden fijo de fases dentro del turno del jugador

El orden de las fases es fijo e inamovible. TurnManager avanza por fases secuencialmente mediante el enum TurnPhase. El jugador no puede atacar antes de moverse.

Fase 1 — Movimiento

El jugador elige moverse o no.

- BFSMovimiento.getCellsInRange() calcula celdas alcanzables con movePoints disponibles.
- JavaFX resalta las celdas alcanzables. El jugador hace clic en su destino.
- Player.move(r,c) ejecuta el movimiento: descuenta movePoints, actualiza posición.
- Si la celda destino es DOOR o STAIRS: TurnManager.changeRoom() gestiona el cambio de sala.
- Si la celda destino tiene item en suelo (requiresAdjacent=false): popup automático recoger/equipar/dejar.
- Si el jugador no se mueve: pasa directamente a Fase 2.

Fase 2 — Recogida (opcional)

Solo aplica si hay un contenedor (requiresAdjacent=true) en celda contigua.

- TurnManager comprueba las 4 celdas adyacentes (N, S, E, O) a la posición actual del jugador.
- Si hay celda con item.requiresAdjacent=true: se ofrece la opción de recoger.
- El jugador elige: recoger y equipar, recoger y guardar en inventario, o ignorar.
- Si no hay contenedor contiguo: fase se salta automáticamente.

■ *Diferencia clave: item en suelo = se pisa y se ofrece (Fase 1). Contenedor = está en celda adyacente y se recoge activamente (Fase 2).*

Fase 3 — Uso de item (opcional)

El jugador puede usar un item de su inventario.

- El jugador selecciona un item del inventario en el panel lateral de JavaFX.
- Item.use(player) ejecuta el efecto: curación, buff, efecto especial.
- Usar un arma en esta fase = equiparla. No cuenta como ataque.
- Solo se puede usar 1 item por turno en esta fase.

Fase 4 — Ataque (opcional)

El jugador puede atacar un enemigo en rango.

- El jugador selecciona un enemigo en rango (resaltado en JavaFX).
- Fórmula de daño: $\text{daño} = \max(0, \text{ataque} * (\text{rand}[0.5-1.5]) - \text{defensa})$. Ver Sección 1.5.
- Si el enemigo muere: Enemy.onDeath() → dropltem si tiene, se elimina de la Cola.
- Solo se puede atacar 1 vez por turno.

1.3 Reglas de movimiento y walkability

Condición	Celdas transitables	Celdas bloqueadas
Tipo de celda	FLOOR, DOOR, DOOR_HIDDEN (si activada), RUNE, LEVER, STAIRS_UP, STAIRS_DOWN	WALL, DOOR_LOCKED (sin llave)
Contenido de celda	cell.unit == null Y cell.item == null O cell.item.requiresAdjacent == false	cell.unit != null (enemigo vivo) O cell.item.requiresAdjacent == true (contenedor)

```
// Cell.java – método central de walkability
public boolean isWalkable() {
    if (type == CellType.WALL || type == CellType.DOOR_LOCKED) return false;
    if (unit != null) return false;
    if (item != null && item.isRequiresAdjacent()) return false;
```

```
return true;
}
```

1.4 Cambio de habitación y escaleras

Tipo de celda	Qué ocurre	Condición previa
DOOR	Dungeon.moveToAdjacentRoom(): actualiza currentRoom. TurnManager carga enemigos de la nueva sala. Se llama onRoomEnter() para diálogos y eventos.	Ninguna.
DOOR_LOCKED	TurnManager comprueba Player.inventory. Si tiene la llave: Cell.type = DOOR. Si no: mensaje de bloqueo, movimiento cancelado.	Player debe tener el item llave en inventario.
DOOR_HIDDEN	Aparece como FLOOR hasta que Room.checkSecretTrigger() la activa. Una vez activa se trata como DOOR normal.	Trigger de activación pisado previamente.
STAIRS_UP/DOWN	Dungeon.changeFloor(): cambia floorLevel, carga grafo del nuevo piso, actualiza currentRoom.	Ninguna.

1.5 Fórmula de combate

La fórmula de daño introduce aleatoriedad controlada para hacer el combate impredecible sin ser injusto. El rango del aleatorio va de 0.5 a 1.5, garantizando que un ataque nunca haga cero daño por mala suerte pura pero sí pueda variar significativamente.

Fórmula oficial de daño

- $\text{vida_defensor} = \text{vida_defensor} - \text{máximo}(0, \text{ataque} * \text{aleatorio} - \text{defensa})$
- $\text{aleatorio} = \text{número aleatorio en el rango } [0.5, 1.5] \text{ (Math.random() * 1.0 + 0.5)}$
- $\text{ataque} = \text{ataque base del atacante} + \text{modificadores por arma} + \text{otros modificadores}$
- $\text{defensa} = \text{defensa base del defensor} + \text{modificadores por armadura} + \text{otros modificadores}$
- $\text{máximo}(0, \dots)$ garantiza que el daño nunca sea negativo (la defensa puede superar al ataque).

```
// CombateManager.java
public int calcularDanio(Unit atacante, Unit defensor) {
    double aleatorio = Math.random() * 1.0 + 0.5; // rango [0.5, 1.5]
    int ataque = atacante.getAtaqueTotal(); // base + arma + modificadores
    int defensa = defensor.getDefensaTotal(); // base + armadura + modificadores
    return Math.max(0, (int)(ataque * aleatorio) - defensa);
}
```

Caso	Aleatorio	Ejemplo (ataque=20, defensa=5)	Resultado
Ataque mínimo	0.5	$\text{máx}(0, 20 \cdot 0.5 - 5) = \text{máx}(0, 5)$	5 de daño
Ataque normal	1.0	$\text{máx}(0, 20 \cdot 1.0 - 5) = \text{máx}(0, 15)$	15 de daño
Ataque crítico	1.5	$\text{máx}(0, 20 \cdot 1.5 - 5) = \text{máx}(0, 25)$	25 de daño
Defensa alta	0.5	$\text{máx}(0, 20 \cdot 0.5 - 15) = \text{máx}(0, -5)$	0 de daño (bloqueado)

1.6 Turno de enemigos y defensa automática

Paso	Qué hace TurnManager
1. Filtrar activos	Seleccionar Enemy cuyo roomId == currentRoom.getId() y hp > 0.
2. executeTurn()	Para cada Enemy: enemy.executeTurn(player, room). El enemigo decide mover y/o atacar según su IA (ver Punto 2).
3. Defensa automática	Cuando Enemy ataca: TurnManager llama a calcularDanio(enemy, player). Player.takeDamage(daño). No requiere acción del jugador.
4. MalacharAlly (Zona 5)	executeAllyTurn(player, room). Actúa después del jugador y antes de los enemigos. Acciones diferenciadas en el log.
5. Log	addLog() registra cada acción. Formato: '[Turno X] Warrior ataca a Kael por 8 de daño.'

1.7 Contadores de turno y condiciones de fin

Contador	Dónde	Cuándo decrementa	Derrota si
Turnos globales	TurnManager.globalTurnsLeft	Siempre al final de cada turno completo.	globalTurnsLeft == 0
Turnos de sala	Room.roomTurnsLeft	Solo si Room.hasRoomTimer == true. Salas de exploración no decrementan.	roomTurnsLeft == 0

Condición	Resultado	Quién detecta
HP jugador <= 0	Derrota	TurnManager tras takeDamage
globalTurnsLeft == 0	Derrota	TurnManager fin de turno
roomTurnsLeft == 0	Derrota	TurnManager fin de turno
Todos los enemigos de sala muertos	Sala despejada	TurnManager tras onDeath
ParasitoEnemy muerto (Zona 5)	Victoria — triggerEnding	TurnManager

1.8 Estructura de TurnManager

```
public class TurnManager {
    private Cola turnQueue; // orden de actuación
    private LSE log; // historial de acciones
```

```

private GameState gameState; // estado serializable
private int globalTurnsLeft; // contador global
private TurnPhase currentPhase; // fase actual del turno

public enum TurnPhase {
    PLAYER_MOVE, PLAYER_PICKUP, PLAYER_USE_ITEM, PLAYER_ATTACK,
    ENEMY_TURNS, CHECK_CONDITIONS, TURN_END
}

public void nextTurn() { ... }
public void addLog(String msg) { ... }
public void onRoomEnter() { ... }
public void changeRoom(String dir) { ... }
public void triggerEnding(CharacterType t) { ... }
public void reconstruirReferencias() { ... }
}

```

1.9 División de trabajo

Miembro	Bloque	Qué incluye
Marino	Motor del juego + integración	Modelo completo (Fase 1), lógica del juego (Fase 2: BFS, TurnManager, combate, acertijos), integración final con JavaFX.
Pacheco	Persistencia + tests	Fase 3 (LectorJSON, GameState, serialización plana). Tests JUnit para todas las clases del modelo desde el día 1.
Castro	Interfaz JavaFX	Fase 4: GridPane, paneles, selección de personaje, diálogos, pantallas de final. Empieza en día 10.

Regla de oro para Castro (JavaFX)

- Los Controllers de JavaFX NO pueden contener lógica de juego.
- Un Controller solo llama a métodos de TurnManager o Player — nunca calcula daño ni mueve unidades.
- Si el método devuelve un int o modifica un atributo del modelo, no va en el Controller.
- Antes de que empiece: sesión de 30 minutos explicando qué puede y qué no puede hacer un Controller.